

[HPC](#)[Running jobs \(Slurm\)](#)[Advanced Slurm](#)

Advanced Slurm

Slurm environments variables

When a job scheduled by Slurm starts, it needs to know certain information about how it was scheduled, etc. E.g., what is its working directory, or what nodes were allocated for it. Slurm passes this information to the job via environmental variables.

The following is a list of commonly used variables that are set by Slurm for each job.

Variable	Description
<code>\$SLURM_JOB_ID</code>	The Job ID
<code>\$SLURM_SUBMIT_DIR</code>	The path of the job submission directory
<code>\$SLURM_SUBMIT_HOST</code>	The hostname of the node used for job submission
<code>\$SLURM_JOB_NODELIST</code>	Contains the definition (list) of the nodes that is assigned to the job
<code>\$SLURM_CPUS_PER_TASK</code>	Number of CPUs per task
<code>\$SLURM_CPUS_ON_NODE</code>	Number of CPUs on the allocated node
<code>\$SLURM_JOB_CPUS_PER_NODE</code>	Count of processors available to the job on this node
<code>\$SLURM_CPUS_PER_GPU</code>	Number of CPUs requested per allocated GPU
<code>\$SLURM_MEM_PER_CPU</code>	Memory per CPU. Same as <code>--mem-per-cpu</code>
<code>\$SLURM_MEM_PER_GPU</code>	Memory per GPU

Variable	Description
<code>\$SLURM_MEM_PER_NODE</code>	Memory per node. Same as <code>--mem</code>
<code>\$SLURM_GPUS</code>	Number of GPUs requested
<code>\$SLURM_NTASKS</code>	Same as <code>-n</code> , <code>--ntasks</code> . The number of tasks
<code>\$SLURM_NTASKS_PER_NODE</code>	Number of tasks requested per node
<code>\$SLURM_NTASKS_PER_SOCKET</code>	Number of tasks requested per socket
<code>\$SLURM_NTASKS_PER_CORE</code>	Number of tasks requested per core
<code>\$SLURM_NTASKS_PER_GPU</code>	Number of tasks requested per GPU
<code>\$SLURM_NPROCS</code>	Same as <code>-n</code> , <code>--ntasks</code> . See <code>\$SLURM_NTASKS</code>
<code>\$SLURM_NNODES</code>	Total number of nodes in the job's resource allocation
<code>\$SLURM_TASKS_PER_NODE</code>	Number of tasks to be initiated on each node
<code>\$SLURM_ARRAY_JOB_ID</code>	Job array's master job ID number
<code>\$SLURM_ARRAY_TASK_ID</code>	Job array ID (index) number
<code>\$SLURM_ARRAY_TASK_COUNT</code>	Total number of tasks in a job array
<code>\$SLURM_ARRAY_TASK_MAX</code>	Job array's maximum ID (index) number
<code>\$SLURM_ARRAY_TASK_MIN</code>	Job array's minimum ID (index) number

Helios cluster special variables:

Value	Description
<code>\$GPU_SCRATCH_DIR</code>	Available only on GPU partition. Local Scratch folder based on SSD. This folder is purged on demand
<code>\$JOBSCRATCH</code> or <code>\$TMPDIR</code>	local scratch dir, limited on size. Delete on job exit

Job States

During its lifetime, a job passes through several states. The most common states are `PENDING`, `RUNNING`, `SUSPENDED`, `COMPLETING`, and `COMPLETED`.

State	Description
PD	Pending. Job is waiting for resource allocation
R	Running. Job has an allocation and is running
S	Suspended. Execution has been suspended and resources have been released for other jobs
CA	Cancelled. Job was explicitly cancelled by the user or the system administrator
CG	Completing. Job is in the process of completing. Some processes on some nodes may still be active
CD	Completed. Job has terminated all processes on all nodes with an exit code of zero
F	Failed. Job has terminated with non-zero exit code or other failure condition

A job may still in pending state for a while. The main reasons are:

(Resources) the job is waiting for resources to become available so that the jobs resource

request can be fulfilled.

(Priority) the job is not allowed to run because at least one higher prioritized job is waiting for resources.

(Dependency) the job is waiting for another job to finish first (`--dependency=... option`).

(DependencyNeverSatisfied) the job is waiting for a dependency that can never be satisfied. Such a job will remain pending forever. Please cancel such jobs.

(QOSMaxCpuPerUserLimit) the job is not allowed to start because your currently running jobs consume all allowed CPU resources for your user in a specific partition. Wait for jobs to finish.

Job Monitoring

Use the `squeue` command to get a high-level overview of all active (running and pending) jobs in the cluster.

Use the `scontrol` command to show more detailed information about a job. For example: `$ scontrol show job 500`

Use `jobinfo` or `seff` to view information about running/completed job.
For example:

```
$ jobinfo 1863
Name           : 15_15_100
User           : xxxxx
Account        : yyyyy
Partition      : mpi
Nodes          : node4-[13-14]
Cores          : 48
GPUs           : 0
State          : COMPLETED
ExitCode       : 0:0
Submit         : 2022-01-09T00:58:21
Start          : 2022-01-09T23:43:24
End            : 2022-01-10T08:02:16
Waited         : 22:45:03
```

```
Reserved walltime   : 1-00:00:00
Used walltime       : 08:18:52
Used CPU time       : 16-15:00:06
% User (Computation): 98.91%
% System (I/O)      : 1.09%
Mem reserved        : 3872M/core
Max Mem used        : 57.63M (node4-13)
Max Disk Write      : 1.66G (node4-13)
Max Disk Read       : 13.95M (node4-13)
```

```
$ seff 1863
Job ID: 1863
Cluster: mesobfc
User/Group: xxxx/yyyyy
State: COMPLETED (exit code 0)
Nodes: 2
Cores per node: 24
CPU Utilized: 16-15:00:06
CPU Efficiency: 99.98% of 16-15:05:36 core-walltime
Job Wall-clock time: 08:18:52
Memory Utilized: 2.70 GB (estimated maximum)
Memory Efficiency: 1.49% of 181.50 GB (3.78 GB/core)
```

To use linux commands (`ps`, `top`...) to monitor your jobs, you can connect to nodes using `ssh`.

This works only for nodes you have jobs running on them. When the job is finished, you will be kicked out of the node.

Job accounting data

After the job exits from the queue, the `sacct` command helps to report the job statistics. For example

```
$ sacct -j 10889
```

JobID	JobName	Partition	Account	AllocCPUS	State	ExitCode
10889	nf-shortV+	smp	xxx	6	COMPLETED	0:0

10889.batch	batch	xxx	6	COMPLETED	0:0
10889.extern	extern	xxx	6	COMPLETED	0:0

By default, the `sacct` command will only bring up information about the user's job from the current day. By using the `--starttime` flag the command will look further back to the given date e.g. :

```
sacct --user=$USER --starttime=YYYY-MM-DD
```

`--format` flag can be used to choose the command output:

```
sacct --user=$USER --format=var_1,var_2, ... ,var_N
```

Variable	Description
Account	The account the job ran under.
AveCPU	Average (system + user) CPU time of all tasks in job.
AveRSS	Average resident set size of all tasks in job.
AveVMSize	Average Virtual Memory size of all tasks in job.
CPUTime	Formatted (Elapsed time * CPU) count used by a job or step.
Elapsed	Jobs elapsed time formatted as DD-HH:MM:SS.
ExitCode	The exit code returned by the job script or salloc.
JobID	The id of the Job.
JobName	The name of the Job.
MaxRSS	Maximum resident set size of all tasks in job.

Variable	Description
MaxVMSize	Maximum Virtual Memory size of all tasks in job.
MaxDiskRead	Maximum number of bytes read by all tasks in the job.
MaxDiskWrite	Maximum number of bytes written by all tasks in the job.
ReqCPUS	Requested number of CPUs.
ReqMem	Requested amount of memory.
ReqNodes	Requested number of nodes.
NCPUS	The number of CPUs used in a job.
NNodes	The number of nodes used in a job.
User	The username of the person who ran the job.

A full list of variables for the `--format` flag can be found with the `--helpformat`

Job dependencies

A job can be given the constraint that it only starts after another job has finished.

In the following example, we have two Jobs, `A` and `B`. We want `Job B` to start after `Job A` has successfully completed.

First we start `Job A` by submitting it via sbatch:

```
sbatch jobA.sh
```

Making note of the assigned `job ID` for `Job A`, we then submit `Job B` with the added condition that it only starts after `Job A` has successfully completed:

```
sbatch --dependency=afterok:jobID_A jobB.sh
```

If we want `Job B` to start after several other Jobs have completed, we can specify additional jobs, using a `:` as a delimiter:

```
sbatch --dependency=afterok:jobID_A:jobID_C:jobID_D jobB.sh
```

We can also tell slurm to run `Job B`, even if `Job A` fails, like so:

```
sbatch --dependency=afterany:jobID_A jobB.sh
```

**TIP**

One can use `-d` option instead of `--dependency`